V:N:M Sparsity for NEAT-Evolved Convolutional Networks

Evolving CNNs with CoDeepNEAT and compressing them with VENOM's V:N:M format on CIFAR-10, CIFAR-100 and ImageNet-100

Final project report

October 2026

Abstract

VENOM's V:N:M format extends the 2:4 sparsity of NVIDIA Sparse Tensor Cores to arbitrary sparsity levels. It has so far been demonstrated on transformers. This project asks whether it also works on convolutional networks, and specifically on networks found by neuro-evolution. We evolve CNNs with CoDeepNEAT on CIFAR-10, CIFAR-100 and ImageNet-100. We retrain the winners with a modern recipe and prune every convolution to V:2:M ($M \in \{4, 8, 16, 32\}$, $V \in \{8, \ldots, 128\}$) with three criteria: magnitude, one-shot OBS and gradual OBS. After a masked, distilled fine-tune we measure accuracy, stored size, and real inference time with VENOM's Spatha kernel.

Result. V:N:M works on NEAT-evolved CNNs.

- 2:4 (3× conv compression) is near-lossless on every well-trained model: median drop 0.3 pt, at most 1.05 pt.
- The best evolved model (CIFAR-100, 76.5% dense, 33.7 MB) keeps 74.4% at 2:8 (5.4 MB) and 70.5% at 2:16 (2.8 MB).
- At 2:32 (1.4 MB, 25×) it keeps 63.7% with gradual second-order pruning, 9.2 pt more than magnitude pruning on average.
- On CIFAR-10, the winner of an 80-generation search (11.1M parameters, 44.4 MB) keeps its accuracy at 2:8 (86.5% vs. 86.5% for the identically fine-tuned dense model, 7.0 MB) and loses 1 pt at 2:16 (85.5%, 3.5 MB). An earlier 8-epoch recipe had made 2:16 look unusable (26%).

Two new observations concern how evolved networks meet V:N:M.

- The search space decides whether compression pays off. CoDeepNEAT's flatten+linear head leaves up to 77% of the bytes unprunable. A residual-sum + global-average-pool space moves over 99% of the bytes into prunable convolutions.
- Evolved channel widths are irregular, so large V forces row padding, which cuts 2:8 compression from 6.3× to as low as 4.1×.

On speed, Spatha accelerates the conv GEMMs by 1.4–2.6× at 2:8 and more at higher M. A full forward pass, however, gains only 1.0–1.37×, because im2col and the non-conv layers are untouched. V:N:M on CNNs is therefore, today, mainly a *compression* result. We also found that CoDeepNEAT's evaluator overstates accuracy by $n/(n-1)$ for n test batches; affected numbers are corrected and the bug is fixed.

Contents

1 Introduction **3**
- 1.1 Research question . . . 3
- 1.2 Contributions . . . 3

2 Background **3**
- 2.1 CoDeepNEAT . . . 3
- 2.2 V:N:M sparsity and Spatha . . . 4
- 2.3 Pruning criteria . . . 4

3 How V:N:M was applied to CNNs **4**
- 3.1 Convolution as a GEMM . . . 4
- 3.2 Format, packing and byte accounting . . . 5
- 3.3 Choosing the mask: three criteria . . . 5

3.4 Recovering accuracy: masked fine-tune with distillation . . . 6
3.5 Running pruned convolutions with Spatha . . . 6

4 Experimental setup **7**
4.1 Pipeline . . . 7
4.2 Hardware, software, data . . . 7
4.3 A bug in CoDeepNEAT's evaluator, and how we corrected for it . . . 8
4.4 Searches . . . 8
4.5 Training recipes . . . 8
4.6 Experiment log . . . 8

5 Results **9**
5.1 The searches and the evolved architectures . . . 9
5.2 Dense accuracy . . . 10
5.3 CIFAR-100: accuracy versus compression . . . 11
5.4 Stored size: why the search space mattered . . . 12
5.5 Pruning criterion: magnitude vs. one-shot OBS vs. gradual OBS . . . 12
5.6 Effect of V and the padding cost of evolved widths . . . 14
5.7 CIFAR-10: the strong search and its winner . . . 14
5.8 ImageNet-100 . . . 16
5.9 Speed . . . 16
5.10 Comparison with hand-designed ResNets (brief) . . . 18

6 Was V:N:M useful? **19**

7 Novel findings **19**

8 Limitations **21**

9 Future work **21**

10 Conclusion **21**

A Per-run comparison of pruning criteria **22**

B Accuracy by V (CIFAR-100, magnitude) **22**

C ResNet results (CIFAR-100, magnitude) **22**

D Earlier runs on under-trained weights **23**

E Changes made to CoDeepNEAT **23**

F Reproduction **23**

1 Introduction

1.1 Research question

CoDeepNEAT [1] evolves CNN topologies by co-evolving small *modules* and *blueprints* that wire modules together. The networks it finds are irregular: odd channel counts, large kernels, multi-way merges. They are also dense and comparatively costly. VENOM [2] introduced the *V:N:M* format, which keeps the hardware-friendly 2:4 pattern of Sparse Tensor Cores but reaches much higher sparsity through an extra vector (column) level. Its Spatha kernel executes the resulting sparse matrix products. The project question is:

> *Can V:N:M sparsity be applied to CNNs, and specifically to NEAT-evolved CNNs? What is the most accurate evolved model we can obtain at a given compression?*

We say V:N:M "works" if three things hold:

(a) accuracy after pruning and fine-tuning stays close to the dense model;

(b) the packed format really reduces the stored model;

(c) Spatha can execute the pruned convolutions, and we measure how fast.

1.2 Contributions

1. **A V:N:M pipeline for CNNs:**
   - conv→GEMM layout, packing and byte accounting;
   - three mask-selection criteria adapted to the V:N:M structure;
   - masked fine-tuning with distillation;
   - a Spatha-based convolution (im2col + sparse SpMM) verified against cuDNN on all 312 benchmark configurations.

   We are not aware of prior V:N:M results on CNNs (Section 8).
2. **Evidence that V:N:M works on NEAT-evolved CNNs** on three datasets, across 10 evolved models and the full $V \times M$ grid, including the winner of an 80-generation CIFAR-10 search.
3. **A V:N:M-friendly change to the CoDeepNEAT search space**: residual-sum aggregation and global average pooling. It raises the dense accuracy of the best evolved model by 4 pt and turns 1.3× whole-model compression into 24×.
4. **Findings specific to evolved networks** (Section 7):
   - the padding cost of irregular widths;
   - the accuracy–speed tension in V;
   - gradual second-order pruning as the decisive factor at high sparsity;
   - why one-shot OBS gains vanish after fine-tuning;
   - why evolved wide, large-kernel convs suit Spatha better than ResNet layers;
   - two silent failure modes of the Spatha kernel;
   - an accuracy bug in CoDeepNEAT's evaluator.

2 Background

2.1 CoDeepNEAT

CoDeepNEAT maintains two populations:

- *modules*: small graphs of conv, BatchNorm, pooling and dropout layers;
- *blueprints*: graphs whose nodes are filled with a module drawn from a module species.

A blueprint plus its sampled modules forms a network, which is scored by validation accuracy after a short training run. NEAT mutations add nodes and connections, and speciation protects new structure. We use the PyTorch implementation by sash-a. In the original code, branches are merged by channel concatenation and the classifier is `flatten→linear`.

2.2 V:N:M sparsity and Spatha

A weight matrix $W \in \mathbb{R}^{R \times K}$ is tiled into blocks of V rows by M columns (Fig. 2).

- **Vector level:** in every block, 4 of the M columns are kept, the same 4 for all V rows.
- **2:4 level:** inside those 4 columns, every row keeps N=2 values.

The density is $2/M$: 2:4, 2:8, 2:16 and 2:32 are 50%, 75%, 87.5% and 93.75% sparse, i.e. 2, 4, 8 and 16× fewer multiply-accumulates.

Spatha [2] computes $Y = W_{\text{sparse}} X$. It gathers the kept columns of X and feeds them to the 2:4 Sparse Tensor Core instruction (`mma.sp`). Spatha is a matrix–matrix (SpMM) library and has no convolution kernel.

2.3 Pruning criteria

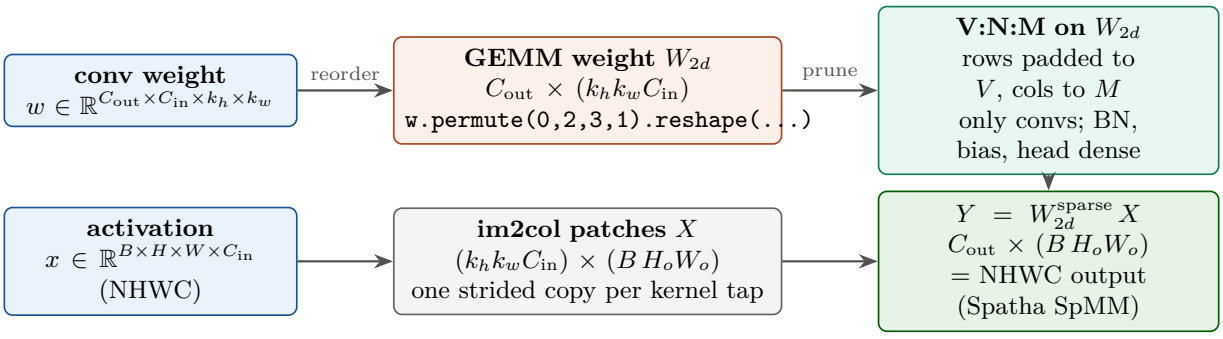
- **Magnitude:** keep the largest weights.
- **Optimal Brain Surgeon (OBS)** [3]: uses the curvature of the layer reconstruction loss, $H = \frac{2}{N} \sum x x^\top$ over the layer inputs x.
  - The cost of removing w_c is $w_c^2 / [H^{-1}]_{cc}$; we rank with the squared form $w_c^2 / [H^{-1}]_{cc}^2$ used by SparseGPT [4].
  - The remaining weights are corrected by $\delta w = -\frac{w_c}{[H^{-1}]_{cc}} H^{-1}_{:,c}$.
- **VENOM's own criterion:** the *gradual* second-order oBERT method [5].

N:M sparsity itself has been studied on CNNs [7, 8], but only at 2:4-like densities that the hardware supports natively.

3 How V:N:M was applied to CNNs

This section describes the parts of the work that are specific to applying a GEMM-oriented sparse format to convolutions.

3.1 Convolution as a GEMM



Column order $(k_h, k_w, C_{\text{in}})$ means one V:N:M column group is a set of M consecutive input channels at one kernel tap: the vector step drops whole (tap, channel) inputs for V output channels at once.

Figure 1: Mapping a convolution onto the matrix that V:N:M prunes and Spatha multiplies.

A convolution with weights $w \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$ is rewritten as

$$W_{2d} = \texttt{w.permute(0,2,3,1).reshape}(C_{\text{out}},\, k_h k_w C_{\text{in}}), \qquad Y = W_{2d}\, X,$$

where X holds the im2col patches with the same $(k_h, k_w, C_{\text{in}})$ column order (Fig. 1). With this order, C_{in} is the fastest-varying index, so an M-column group is M consecutive input channels at one kernel tap. The vector step of V:N:M therefore removes whole (tap, input-channel) pairs for a group of V output channels. CNNs tolerate this kind of structured removal well. Only conv weights are pruned. BatchNorm, biases and the classifier stay dense in fp32.

3.2 Format, packing and byte accounting

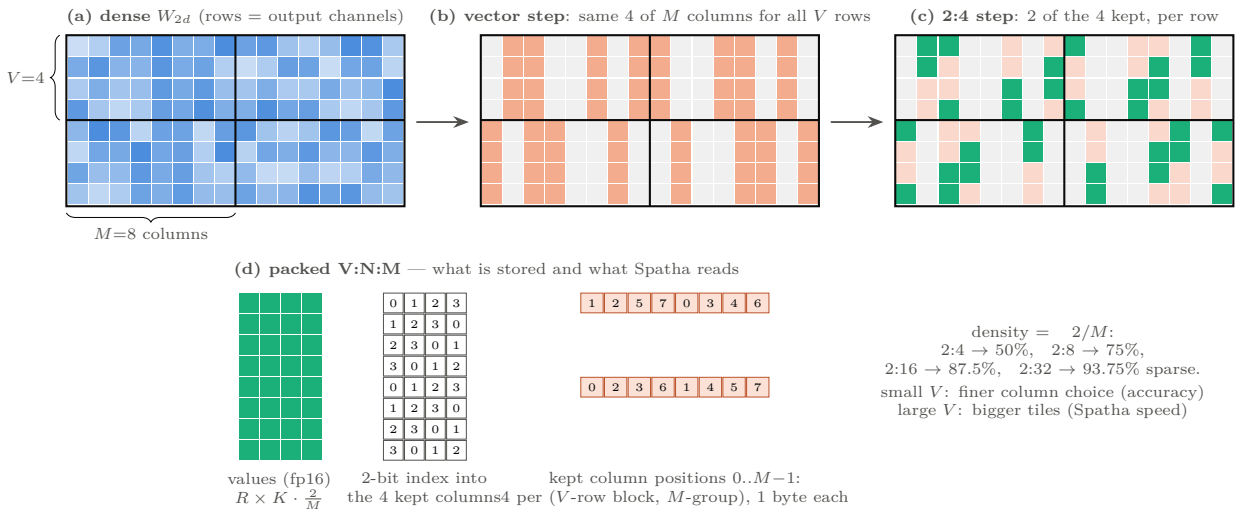


Figure 2: V:N:M on a toy 8×16 matrix with V=4, M=8 (2:8). (b) The vector step keeps 4 columns per block. (c) The 2:4 step keeps 2 of those per row. (d) Stored form: dense fp16 values, a 2-bit index per value, and the kept column ids per block.

`vnm/vnm_pack.py` implements prune / pack / unpack.

- **Padding.** W_{2d} is zero-padded to a multiple of V rows and M columns, and the packed tensors store the padded matrix.
- **Stored bytes** (`packed_nbytes_tight`): 2 fp16 values per row per M-group, a 2-bit index per value, and, per V-row block and M-group, 4 kept-column positions $0..M-1$ at one byte each.
- **Reported compression.** *Conv compression* is fp32 dense conv bytes divided by packed conv bytes. *Stored size* adds the dense fp32 non-conv parameters.
- **Round trip.** `unpack(pack(W))` reproduces the pruned matrix exactly; this is asserted in every run.

Two consequences matter for evolved networks:

- **Metadata.** The column-id overhead shrinks as V grows, because the ids are shared by more rows. On power-of-two layers, larger V therefore *raises* compression; ResNet-18 at 2:8 goes from 6.4× at V=8 to 7.0× at V=128.
- **Row padding.** Evolved layers have channel counts such as 142, 171, 211 and 269. Padding them to a multiple of V wastes storage and compute. bp482 at 2:8 drops from 6.3× at V=8 to 4.1× at V=128 (Section 5.6).

3.3 Choosing the mask: three criteria

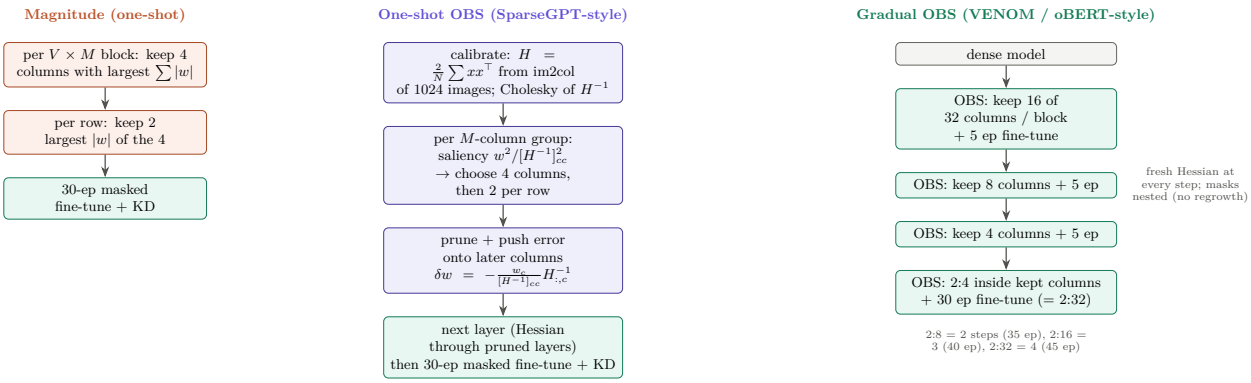


Figure 3: The three mask-selection procedures. All end with the same masked, distilled fine-tune.

Magnitude (one-shot). In each $V \times M$ block, keep the 4 columns with the largest $\sum |w|$ over the block's V rows, then the 2 largest $|w|$ per row among them.

One-shot OBS, adapted to V:N:M (`vnm/prune_obs.py`). A SparseGPT-style layer-wise solver:

1. **Hessian.** $H = \frac{2}{N} \sum xx^\top$ over im2col patches of 1024 training images, with 1% diagonal damping. We take the upper Cholesky factor of H^{-1}.
2. **Mask.** Columns are processed left to right in groups of M. At the start of each group, the V:N:M mask is chosen from the OBS saliency $w^2/[H^{-1}]^2_{cc}$: summed over the V rows to choose 4 columns, then per row to choose 2.
3. **Compensation.** Each pruned column's error is pushed onto the columns not yet processed, with lazy updates in blocks of 128.
4. **Order.** Layers are pruned in module order. Each layer's Hessian is collected through the already-pruned layers before it.

Gradual OBS (VENOM/oBERT-style) (`vnm/prune_gradual.py`). The target is reached in steps. Each step uses a fresh Hessian, an OBS pruning pass and a 5-epoch masked fine-tune:

- first, column (vector) pruning only, keeping $M/2 \to M/4 \to \cdots \to 4$ columns per block;
- then a final 2:4 step inside the kept columns, followed by a 30-epoch fine-tune.

2:8, 2:16 and 2:32 take 2, 3 and 4 steps (35, 40 and 45 fine-tune epochs). Masks are nested: weights removed in an earlier step have zero saliency and never come back.

3.4 Recovering accuracy: masked fine-tune with distillation

After pruning, the network is fine-tuned for 30 epochs:

- AdamW, lr 5×10^{-4}, wd 0.01, 1-epoch warm-up, then cosine;
- knowledge distillation from the dense model (α=0.5, T=4);
- the mask is re-applied after every optimiser step, so the sparsity pattern is exactly preserved.

3.5 Running pruned convolutions with Spatha

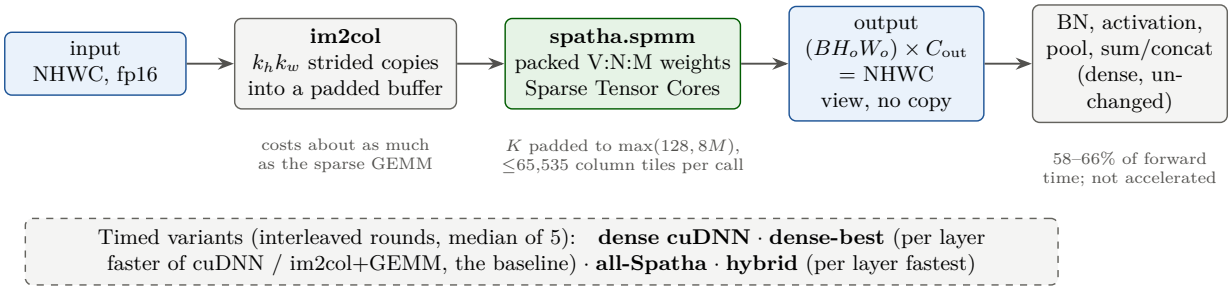


Figure 4: Inference path of a V:N:M convolution (`SpathaConv2d` in `vnm/e2e_bench.py`).

Build. We built Spatha from `UDC-GAC/venom` in a separate environment (PyTorch 1.13.1, CUDA 11.7) for sm_86, which runs on our sm_89 GPUs.

- V equals Spatha's thread-block row tile, and the default build supports $V \in \{64, 128\}$.
- We added builds for V=32 and V=256.
- $V \in \{8, 16\}$ would need a kernel redesign (the warp tile is 32 rows). Those settings have accuracy and compression results but no timing.

Execution. Every `Conv2d` of a pruned network is replaced by a `SpathaConv2d` (Fig. 4):

1. **im2col:** one strided copy per kernel tap writes the NHWC input straight into a padded $(k_h k_w C_{\text{in}}) \times (BH_oW_o)$ buffer. This is 2× faster than `F.unfold` plus a permute.
2. **SpMM:** `spatha.spmm` runs on the packed weights.
3. **Output:** the result, $(BH_oW_o) \times C_{\text{out}}$, is already NHWC and is used as a channels-last view without a copy.

Correctness. Spatha's own sparsifier can pick a different pattern among tied zeros, so we add 10^{-7} noise before conversion to reproduce our mask exactly. Every one of the 312 end-to-end configurations was checked against cuDNN with the same masked weights on real test images: top-1 agreement $\geq 99.2\%$, relative logit error $\leq 2.4 \times 10^{-3}$.

Two silent Spatha failures , found and worked around:

- **(a) Out-of-bounds write at M=32.** When K is an odd multiple of 128, the kernel writes out of bounds without an error, and cuBLAS/cuDNN calls made later crash. Fix: we pad K to a multiple of $\max(128, 8M)$.
- **(b) Column-tile limit.** One call handles at most 65,535 column tiles of 64 (about 4.2M columns). Beyond that it returns garbage without an error, so large inputs must be chunked; the ImageNet-100 check therefore uses 32 images.

4 Experimental setup

4.1 Pipeline

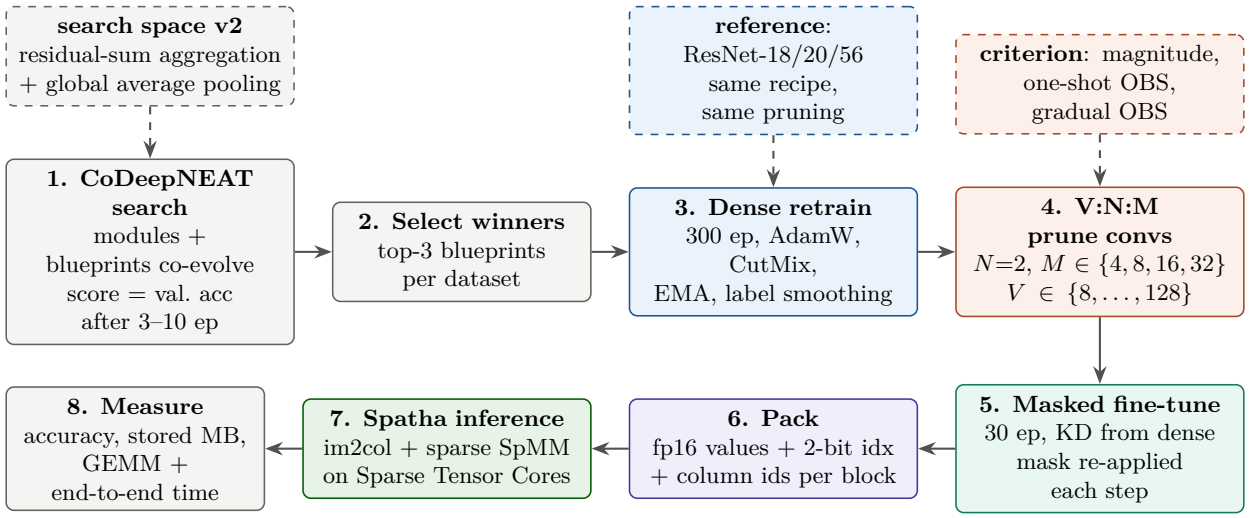


Figure 5: End-to-end pipeline. V:N:M is applied post hoc to the winners of the search. Dashed boxes are the variations we added along the way.

4.2 Hardware, software, data

- **Hardware:** 3× NVIDIA RTX 6000 Ada (sm_89, 48 GB). They were *shared*: other users' jobs often kept them at 100% utilisation. Timings use interleaved rounds and medians to compensate.
- **Software:**
  - search, training and pruning: Python 3.13, PyTorch 2.6, bf16 autocast;
  - Spatha: Python 3.10, PyTorch 1.13.1, CUDA 11.7.
- **Datasets:**
  - CIFAR-10/100: 50k train, 10k test, 32^2.

  - ImageNet-100 (CMC split [6]): 126,689 train images; the 5,000 validation images are the test set. Images are cached at 144 px with random 128^2 crops for training, and resized to 146 with a 128^2 centre crop for testing.
  - Full ImageNet was not available on the machine.

- **Augmentation:**
  - search: none (CoDeepNEAT's evolved augmentation was disabled);
  - retraining and every V:N:M fine-tune: random crop and flip, plus CutMix in recipe v2.

4.3 A bug in CoDeepNEAT's evaluator, and how we corrected for it

CoDeepNEAT's `test_nn` averages the per-batch accuracies and divides the sum by the index of the last batch, i.e. by $n-1$ for n batches. Every accuracy it reports is therefore inflated by $n/(n-1)$: +1.3% relative on a 10k test set at batch 128 (n=79), +2.6% at batch 256 (n=40) and +2.5% on ImageNet-100's 5,000 test images at batch 128. We found it when CoDeepNEAT reported 87.68% for a checkpoint that our own evaluator scores at 85.61%; with the fix, CoDeepNEAT's evaluator gives 85.59%.

- **Not affected:** everything evaluated by our own code (`vnm/recipe.py`): all retrained models, every 30-epoch V:N:M grid, OBS, gradual OBS, ResNets, ImageNet-100 after retraining, all timing. This is every main result.
- **Affected:** numbers produced by CoDeepNEAT's evaluator: its 60-epoch full-train accuracies, the first V:N:M grids (CIFAR-10 small search, early CIFAR-100 and ImageNet-100 rows), and the search fitness. In this report the test-set numbers of the first group are *corrected* by the factor $(n-1)/n$ (exact if every batch had the same accuracy; error $\lesssim$0.1 pt) and marked ^{c}. Search accuracies are left as logged (fitness is only used for ranking, which the factor does not change) and marked ^{s}.
- The evaluator in `final/` is fixed (every sample counts once), so re-running any script gives exact numbers.

4.4 Searches

	CIFAR-10	CIFAR-10 strong	CIFAR-100	ImageNet-100	CIFAR-100 res+GAP
generations	12	**80**	12	10	12
blueprint / module population	20 / 50	25 / 80	20 / 50	10 / 20	20 / 50
evaluations × epochs per blueprint	2×3	4×12	2×3	2×3	$2\times$**10**
batch / conv channels	128 / 32–256	256 / 64–512	128 / 32–256	128 / 32–128	128 / 32–**384**
module species	4	6	4	4	4
aggregation	concat	concat	concat	concat	**residual sum**
classifier head	flatten+linear	flatten+linear	flatten+linear	flatten+linear	**GAP+linear**
add-node / add-conn. rate (bp / module)	default	0.16, 0.12 / 0.1	default	default	0.3 / 0.25
CoDeepNEAT full train of top 3	60 ep	150 ep, fm 1 and 3	60 ep	40 ep	60 ep
wall-clock	1.7 h (3 GPUs)	about 2 days (2 GPUs)	1.7 h	2.4 h	12.8 h

Table 1: Search configurations (`CoDeepNEAT/configuration/configs/*.json`; the strong CIFAR-10 search is `ada_cifar10_strong`). "fm" is CoDeepNEAT's feature multiplier, which rebuilds the same genotype with every channel count scaled so that the parameter count grows by that factor.

4.5 Training recipes

recipe v1	AdamW, lr 2×10^{-3}, wd 0.05, 3-epoch warm-up + cosine, label smoothing 0.1, batch 128, crop + flip, bf16; 200 epochs (CIFAR-100), 120 epochs (ImageNet-100)
recipe v2	v1 + **300 epochs**, CutMix (p=0.5, α=1), EMA of weights (0.999; the better of live and EMA weights is kept)
V:N:M fine-tune	AdamW, lr 5×10^{-4}, wd 0.01, 30 epochs, KD (α=0.5, T=4), mask re-applied after every step

Table 2: Recipes (`vnm/recipe.py`). The evolved architectures are kept unchanged; only their weights are retrained from scratch.

4.6 Experiment log

Each experiment was motivated by the result of the previous one.

#	experiment	outcome / why the next step
E1	Build Spatha for Ada; V:N:M pack/prune code; GEMM-level benchmark	Spatha correct within fp16 tolerance; $V \in \{32, 64, 128, 256\}$ builds
E2	Search on CIFAR-10, CIFAR-100, ImageNet-100 (orig. space)	winners found; CIFAR-100/IN-100 dense acc. only 42–49% / 28–29% (weak training loop)
E3	First V:N:M grid (magnitude, 8-ep FT)	2:4 lossless on CIFAR-10; higher M unreliable on under-trained models → fix training
E4	Retrain winners with recipe v1 (200 ep)	CIFAR-100 67%, ImageNet-100 52–54%
E5	Full $V \times M$ grid, 30-ep KD FT (CIFAR-100, ImageNet-100)	graceful degradation, V=8 best
E6	Recipe v2 (300 ep, CutMix, EMA) + grid	CIFAR-100 evolved 72.4%
E7	ResNet-18/20/56 from scratch, same recipe + grid	ResNets lead by about 6 pt; evolved heads not compressible → change search space
E8	Residual-sum + GAP search, retrain top 6, grid on top 3	76.5% dense; convs hold >99% of bytes
E9	One-shot OBS (V:N:M-aware)	large zero-shot gain, small gain after FT
E10	Gradual OBS (VENOM-style)	+9.2 pt at 2:32 over magnitude
E11	Spatha GEMM timing on every pruned conv; V-scan on synthetic shapes	1.4–2.6× at 2:8
E12	End-to-end Spatha inference, 12 models × 3 V × 4 M × 2 batch sizes	1.0–1.27× whole-network
E13	Strong CIFAR-10 search (80 generations), CoDeepNEAT full train at fm 1 and 3; 8-epoch magnitude recover on the winner bp1485	2:4 recovers to dense, 2:8 loses about 6 pt, 2:16 fails; found the evaluator bug
E14	Our pipeline on bp1485: 30-ep KD grid, dense control, gradual OBS, Spatha timing	see Section 5.7

Table 3: Experiments in the order they were run.

5 Results

5.1 The searches and the evolved architectures

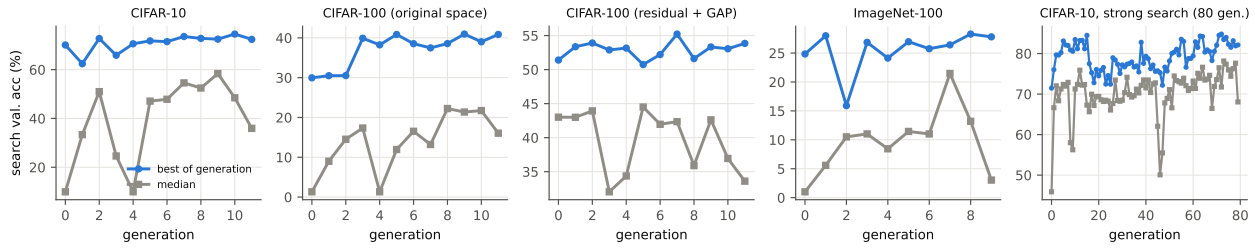


Figure 6: Search fitness (validation accuracy) per generation, as logged by CoDeepNEAT (^{s}, slightly inflated; Section 4.3). First four panels: our searches (40 evaluations per generation; 20 on ImageNet-100). Right: the 80-generation strong CIFAR-10 search (best and median blueprint of each generation). The best of a generation rises by 3–14 pt over a run; the median fluctuates widely because new structure is evaluated after only 3–12 epochs.

search	model	params	convs	kernels	conv widths	bytes in convs
CIFAR-100 res+GAP	bp493	8.42M	10	1, 3, 5, 7	142–342	99.7%
CIFAR-100 res+GAP	bp482	2.51M	6	3, 5	142–342	99.2%
CIFAR-100 res+GAP	bp741	4.33M	5	3, 5, 7	148–291	99.6%
CIFAR-100 orig.	bp430 / bp427	3.62M / 3.27M	10	1, 5	44–457	60% / 66%
CIFAR-100 orig.	bp499	5.49M	7	1, 5	44–457	**23%**

Table 4: Evolved CIFAR-100 winners. All are shallow (5–10 convs) and wide, with large kernels and irregular channel counts. The original space puts most of the bytes into the dense flatten+linear head.

Observation. Scoring candidates after only 3–10 epochs favours shallow, wide, large-kernel networks that converge quickly. This has two consequences: dense accuracy is capped below hand-designed deep networks, but the convolutions are large GEMMs that suit Spatha (Section 5.9).

5.2 Dense accuracy

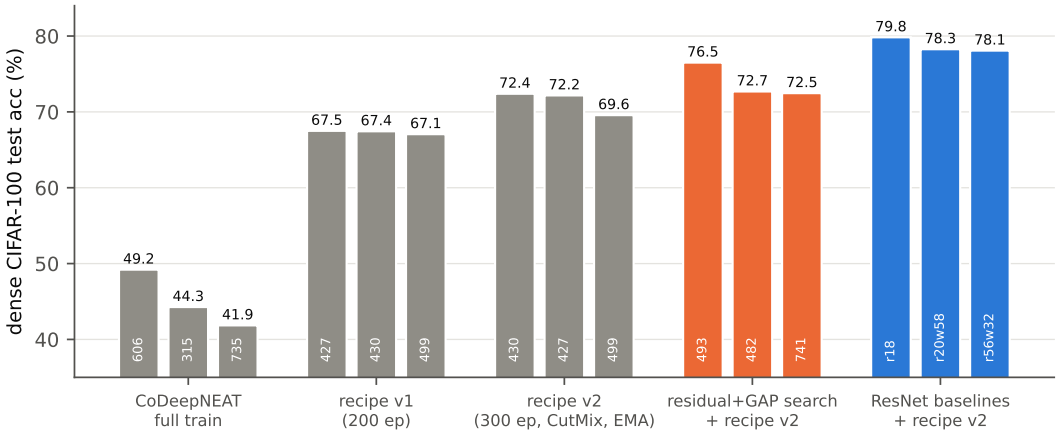


Figure 7: Dense CIFAR-100 test accuracy of the top 3 models at each stage. Better training lifts the same evolved architectures from at most 50% to 72%. The residual+GAP search adds 4 pt more at the top.

dataset	model	params	search acc.^{s}	final dense acc.
CIFAR-10	evolved bp495 / bp671 / bp546 (small search)	1.91M / 0.57M / 2.84M	–	80.60 / 79.73 / 78.26^{c}
CIFAR-10	evolved bp1485, fm 3 (strong search)	11.11M	84.5^{s}	**85.61**
CIFAR-100	evolved bp430 / bp427 / bp499 (recipe v2)	3.62M / 3.27M / 5.49M	38.5 / 38.3 / 37.5	72.39 / 72.17 / 69.57
CIFAR-100	evolved res+GAP bp493 / bp482 / bp741 (v2)	8.42M / 2.51M / 4.33M	55.2 / 54.2 / 53.9	**76.50** / 72.69 / 72.47
ImageNet-100	evolved bp207 / bp182 / bp230 (v1, 120 ep)	2.77M / 3.05M / 0.47M	25.7 / 26.0 / 28.3	53.68 / 52.36 / 52.28
CIFAR-100	ResNet-18 / ResNet-20 (w58) / ResNet-56 (w32)	11.22M / 3.58M / 3.42M	–	79.83 / 78.26 / 78.08

Table 5: Dense models that were pruned. CIFAR-10 models use CoDeepNEAT's own full-train weights (60 epochs; 150 epochs at fm 3 for bp1485). ^{c} corrected for the evaluator bug; ^{s} search fitness as logged (Section 4.3).

5.3 CIFAR-100: accuracy versus compression

family	criterion	model	params	dense	2:4 (3.1×)	2:8 (6.3×)	2:16 (12.6×)	2:32 (25×)
evolved res+GAP	magnitude	bp493	8.42M	76.50	76.26	74.01	70.06	57.23
	OBS one-shot	bp493			–	73.92	69.23	57.98
	OBS gradual	bp493			–	**74.40**	**70.50**	**63.72**
evolved res+GAP	magnitude	bp741	4.33M	72.47	72.20	70.15	66.60	57.05
	OBS one-shot	bp741			–	70.45	66.43	57.11
	OBS gradual	bp741			–	**70.92**	**68.09**	**61.78**
evolved res+GAP	magnitude	bp482	2.51M	72.69	72.44	69.60	62.14	42.37
	OBS one-shot	bp482			–	69.94	62.75	47.23
	OBS gradual	bp482			–	**70.24**	**64.33**	**55.37**
evolved (orig.)	magnitude	bp430	3.62M	72.39	71.34	68.40	62.40	49.79
evolved (orig.)	magnitude	bp427	3.27M	72.17	71.50	68.61	62.35	50.50
evolved (orig.)	magnitude	bp499	5.49M	69.57	69.29	66.84	63.42	56.53

Table 6: CIFAR-100, evolved models (recipe v2), 30-epoch KD fine-tune after pruning (35–45 epochs for gradual OBS). Cells: test accuracy (%), best over $V \in \{8, \ldots, 128\}$; the best V is almost always 8. Shaded: best evolved result per column. Compression is conv-only and nearly identical across models at a given M.

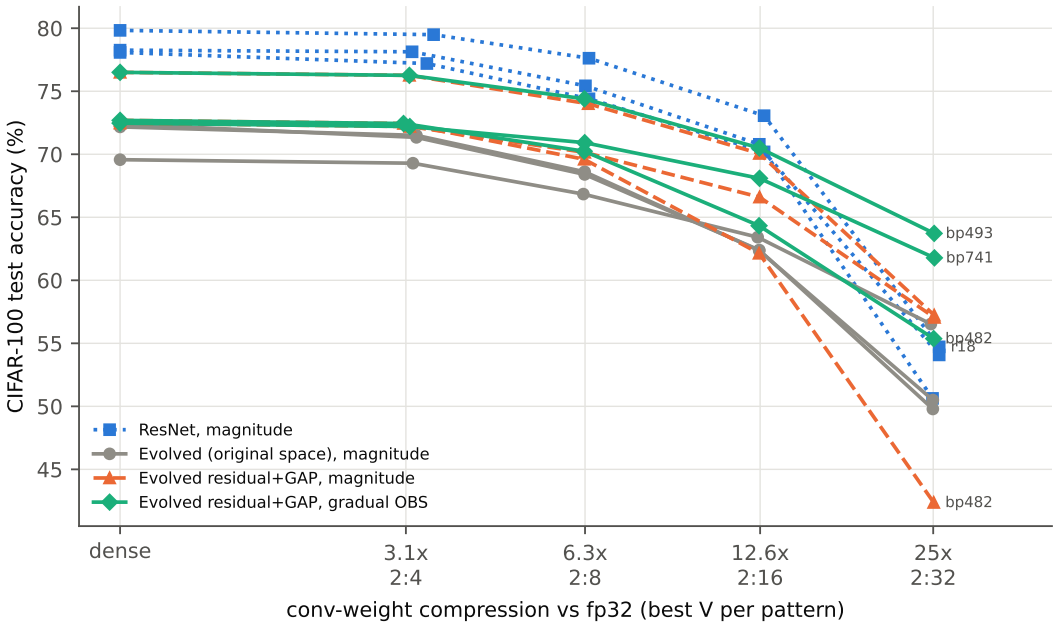


Figure 8: CIFAR-100 accuracy against conv compression (best V per pattern). Gradual OBS (aqua) lifts the evolved residual models above every magnitude-pruned curve at 25×, including the ResNets (blue, dotted).

V:N:M works on evolved CNNs. 2:4 costs 0.24–1.05 pt. 2:8 costs 2.3–4 pt with magnitude pruning and 1.6–2.5 pt with gradual OBS. 2:16 costs 5.9–10.6 pt with magnitude pruning and 4.4–8.4 pt with gradual OBS, for 8× fewer MACs. The degradation is smooth, with no cliff until 2:32 under magnitude pruning.

5.4 Stored size: why the search space mattered

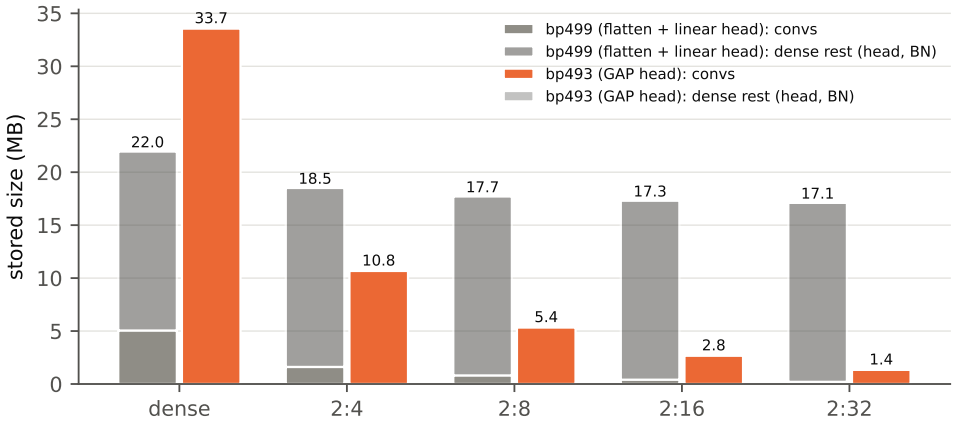


Figure 9: Stored size at V=8. bp499 (original space) keeps a 17 MB dense flatten+linear head, so even 2:32 only shrinks it 1.3×. bp493 (GAP head) shrinks 33.7 MB → 10.8 / 5.4 / 2.8 / 1.4 MB.

In the original search space, conv compression does not become model compression. After we added residual-sum aggregation and a GAP head, over 99% of the bytes sit in convolutions and the whole model shrinks by the conv ratio. Figure 10 shows every CIFAR-100 configuration we trained on the size–accuracy plane. Below about 1 MB, the frontier consists of gradual-OBS evolved models.

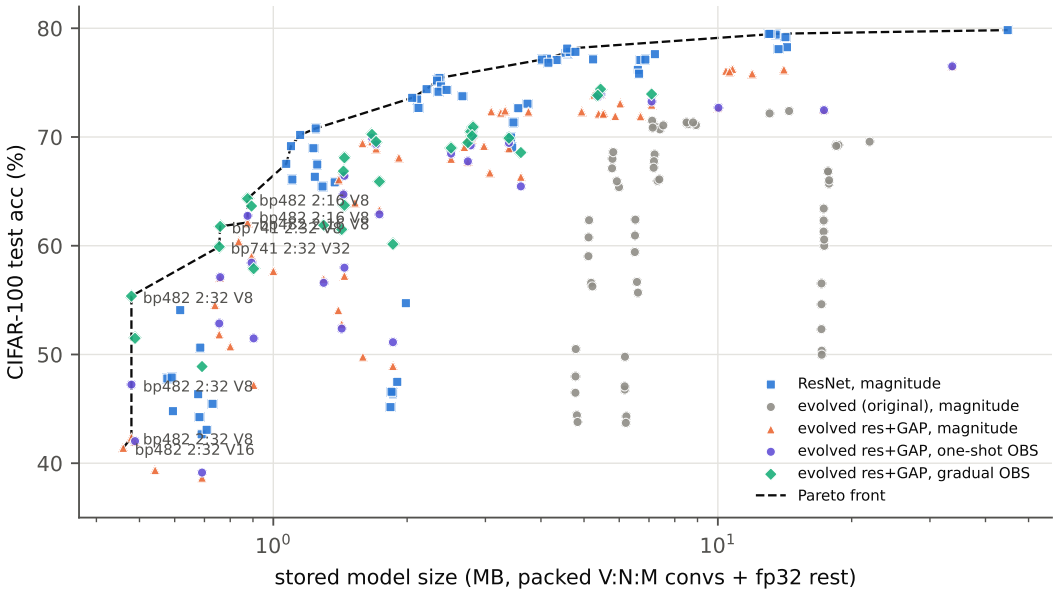


Figure 10: All CIFAR-100 runs (every model, V, M and criterion): accuracy against stored size. The dashed line is the Pareto front. Above about 2 MB, the size-matched ResNets dominate. At the smallest sizes, the gradual-OBS evolved models form the front.

5.5 Pruning criterion: magnitude vs. one-shot OBS vs. gradual OBS

pattern	gradual − magnitude	gradual − one-shot	one-shot − magnitude	gradual beats magnitude
2:8	+0.62	+0.37	+0.26	8 / 9
2:16	+2.48	+2.78	−0.30	9 / 9
2:32	**+9.18**	+7.71	+1.48	9 / 9

Table 7: Mean accuracy difference after fine-tuning (points), over 3 evolved residual models × $V \in \{8, 32, 128\}$. Per-run values are in Appendix A.

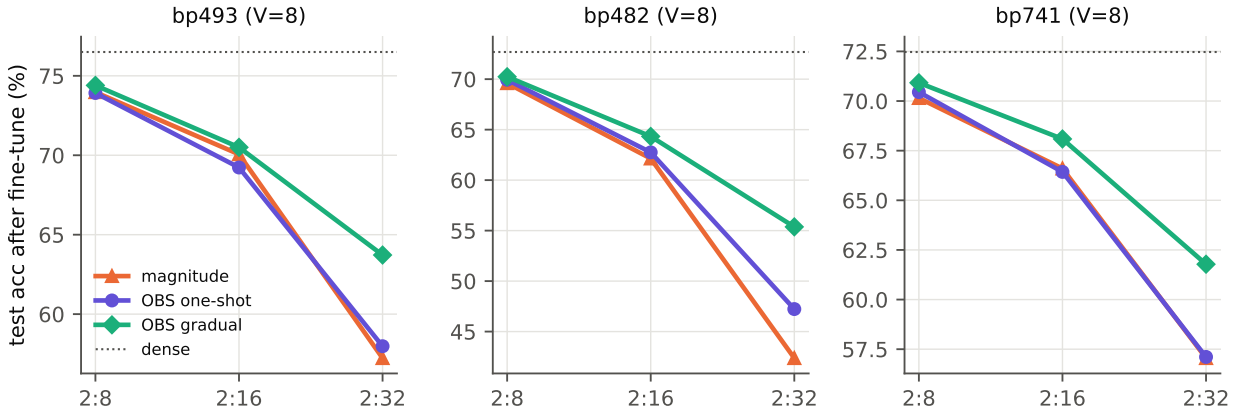


Figure 11: Fine-tuned accuracy of the three criteria at V=8. Dotted line: dense model.

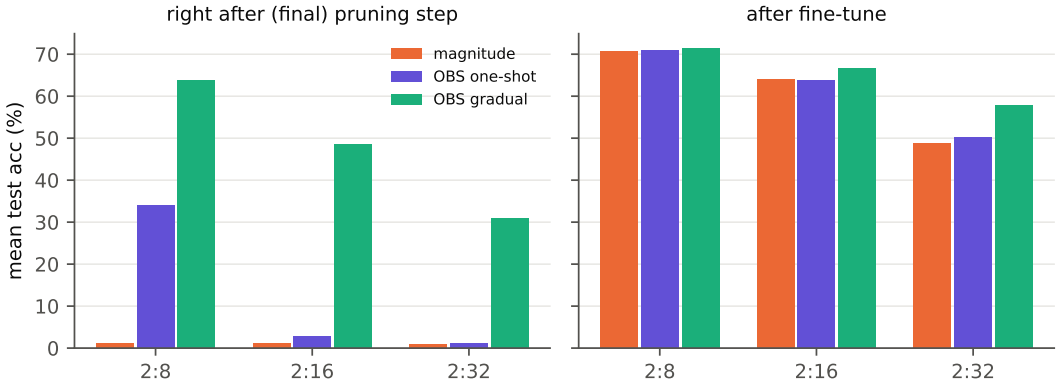


Figure 12: Left: accuracy right after the (final) pruning step. Right: after fine-tuning. One-shot OBS leaves a far better starting point than magnitude pruning: at 2:8, 22–49% instead of about 1%. Fine-tuning erases that advantage, while the gradual gain persists and grows with sparsity.

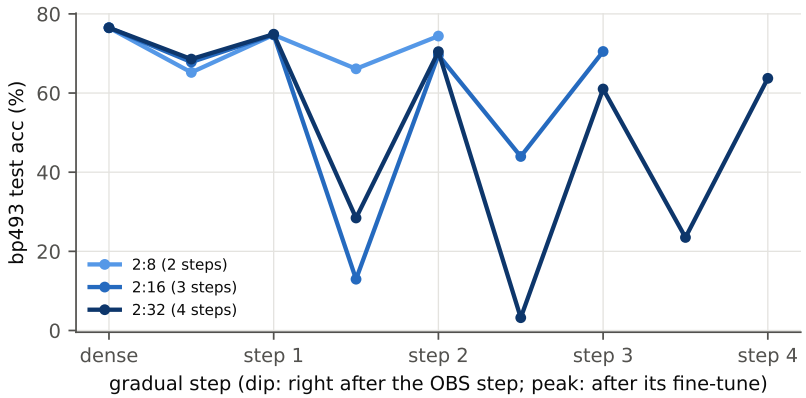


Figure 13: Gradual OBS on bp493 (V=8). The first column step (half the columns removed) costs only about 2 pt after 5 epochs. Most of the damage happens in the last column steps, and the final 2:4 step plus its 30-epoch fine-tune *recovers* accuracy (2:32: 61.0 → 63.7%).

At high sparsity, the criterion matters more than the architecture. With magnitude pruning, 2:32 collapses to 42–57% for every family. Gradual OBS adds 5–13 pt and gives the best 2:32 result in the study, 63.7% at 25×. The gain comes from moving into the sparse subspace *gradually* and re-estimating curvature along the way, which is why VENOM uses gradual oBERT. A good one-shot starting point is not enough.

5.6 Effect of V and the padding cost of evolved widths

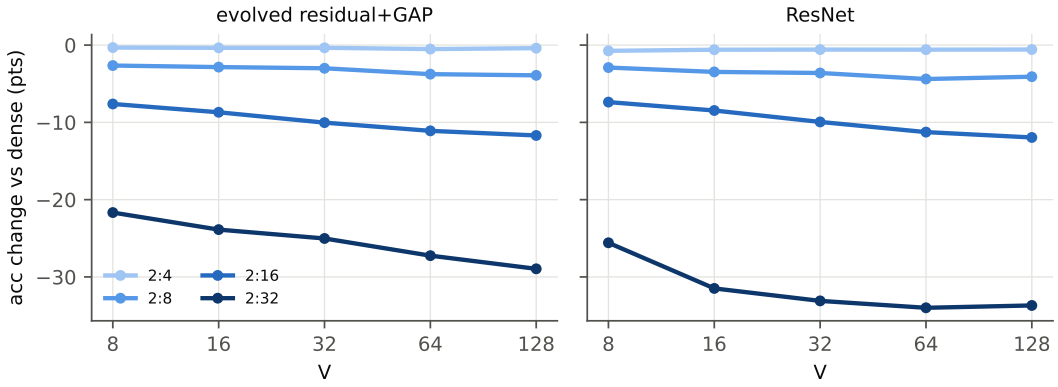


Figure 14: Mean accuracy change against dense as a function of V (magnitude pruning, 3 models each). Smaller V is better at every M, and the effect grows with M. At 2:4, V hardly matters (spread at most 0.8 pt).

Smaller V means the 4 kept columns are chosen for fewer rows, so the mask is closer to unstructured 2:M. The accuracy cost of V=128 over V=8 is 1–3 pt at 2:8 and 3–5 pt at 2:16. V also changes *compression*, and here evolved networks behave differently from hand-designed ones (Fig. 15).

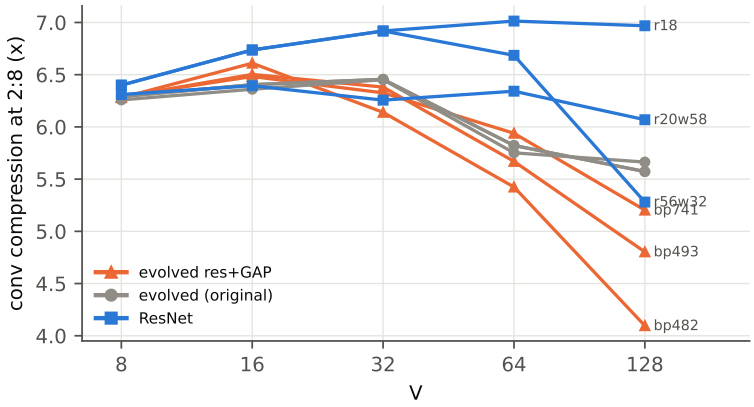


Figure 15: Conv compression at 2:8 as a function of V. **ResNets:** power-of-two widths, so larger V amortises the column ids and compression rises up to V=64–128. **Evolved networks:** irregular widths (142, 171, 211, 269, ...) are padded up to V rows, so compression falls from 6.3× to 4.1–5.7× at V=128.

5.7 CIFAR-10: the strong search and its winner

Our own CIFAR-10 search (12 generations, 3-epoch evaluations) was short. A second, much longer CIFAR-10 search was run on the same machine (`ada_cifar10_strong`, Table 1): 80 generations, 4 evaluations of 12 epochs per blueprint, wider channel range, followed by CoDeepNEAT's 150-epoch full train of the top 3 at feature multiplier 1 and 3. We take its winner through our whole pipeline.

Search and winner. The best-of-generation fitness rises from 71% to about 85% over the 80 generations (Fig. 6, right). The winner, blueprint 1485, was only third in the evolution ranking but first after full training (Table 8). Its three blueprint nodes all use module 1463 (species 19): a narrowing 3×3 conv with BN, dropout 0.1 and LeakyReLU, then a widening 3×3 conv with 3×3/stride-3 max-pooling, dropout 0.37 and ReLU6. A skip from the stem module is concatenated with a side branch (Fig. 16). At feature multiplier 3 the network has six convs with 329 / 624 channels, a 1248-channel concatenation, and 11.1M parameters. Because the max-pools shrink the map to 4×4, the flatten+linear head is tiny: **99.9% of the bytes are convolutions**, so this model needed no change of search space to be compressible.

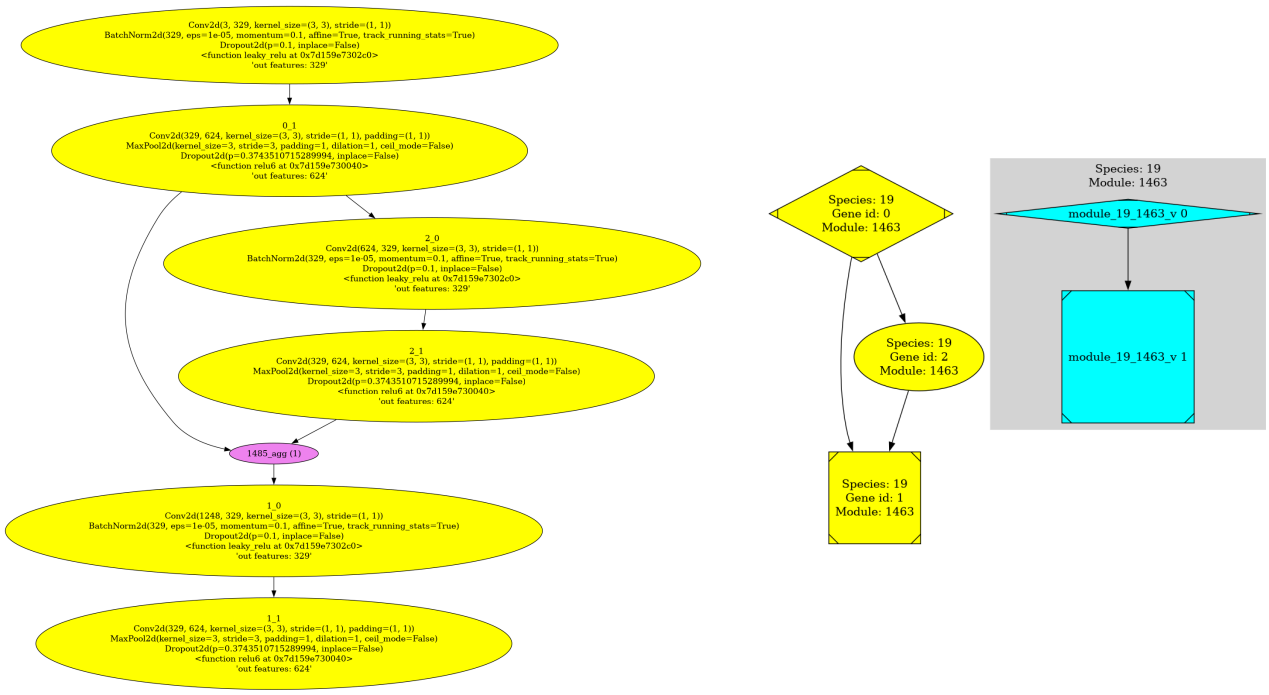


Figure 16: bp1485 drawn by CoDeepNEAT's own visualisers. Left: phenotype at feature multiplier 3 (yellow: module layers; violet: concatenation). Right: genotype; all three blueprint nodes point at species 19 / module 1463.

model (fm 3)	evolution rank	CoDeepNEAT full train (150 ep)[c]	saved checkpoint, exact
bp6826	1	85.45	–
bp6724	2	84.42	–
bp1485	3	85.98 (peak)	**85.61**

Table 8: Full training of the top 3. [c] CoDeepNEAT reported 87.64 / 86.58 / 88.18%; corrected for the evaluator bug (Section 4.3). The pruning experiments start from the saved checkpoint, which our evaluator scores at 85.61%.

V:N:M results. Table 9 compares three ways of recovering accuracy after pruning bp1485. The first is the original study's recipe: magnitude pruning at V=8, then 8 epochs of Adam (lr 10^{-4}) with the mask re-applied, no augmentation and no distillation. Re-run with the fixed evaluator it reproduces the reported numbers once corrected (2:4 86.0%, 2:8 79.8%, 2:16 26.4%; reported 88.29 / 81.72 / 23.65% under the bug). The second is our 30-epoch recipe (crop + flip, distillation from the dense model). Because that recipe also trains the dense model further, we add a **dense control**: the unpruned model fine-tuned with the identical 30-epoch recipe reaches 86.45%, which is the fair reference.

recovery after pruning	V	2:4 (3.2×, 14.0 MB)	2:8 (6.3×, 7.0 MB)	2:16 (12.6×, 3.5 MB)	2:32 (25×, 1.8 MB)
none (zero-shot)	8	41.11	16.53	10.97	10.00
8-ep Adam, no aug., no KD (original study)	8	86.01	79.78	26.36	–
30-ep KD fine-tune, magnitude	8	86.30	**86.52**	**85.48**	79.11
30-ep KD fine-tune, magnitude	32	**86.35**	86.36	85.02	80.52
gradual OBS	8	–	–	84.42	**81.55**
gradual OBS	32	–	–	83.33	80.35
dense checkpoint 85.61 % (44.4 MB); **dense control**, same 30-ep fine-tune without pruning: **86.45** %					

Table 9: CIFAR-10, bp1485 (11.1M parameters, fm 3). Test accuracy (%), all with the fixed evaluator. Sizes are conv bytes packed at V=8 (V=32 is slightly smaller: 13.5 / 6.7 / 3.4 / 1.7 MB), plus 44 KB of dense non-conv parameters.

- **The weak recipe, not V:N:M, limited the original study.** With our fine-tune, 2:8 matches the dense control (86.52 vs 86.45%) and 2:16 is within 1 pt of it (85.48%) at 12.6× compression, where the 8-epoch recipe gave 26%.

- **The best CIFAR-10 result:** a 44.4 MB evolved CNN stored in 3.5 MB at 85.5% (2:16), or in 7.0 MB with no loss (2:8).
- V **hardly matters on this network** (within 0.5 pt at 2:4–2:16, and V=32 is better at 2:32), unlike on the CIFAR-100 models. Its layers are wide (329–1248 channels), so the vector constraint costs little and row padding is small.
- **Gradual OBS** helps only at the highest sparsity: at 2:32 and V=8 it gives 81.55% against 79.11% for magnitude pruning (+2.4 pt), the best 2:32 result on this network. At 2:16, and at V=32, it is 0.2–1.7 pt *below* magnitude pruning. This is the opposite of CIFAR-100, where gradual OBS added 9.2 pt at 2:32: on this wide network with a well-tuned fine-tune, magnitude pruning already leaves little to recover.

5.8 ImageNet-100

model	params	dense	2:4	2:8	2:16	2:32
bp207	2.77M	53.68	**54.30**	53.28	48.00	38.70
bp182	3.05M	52.36	51.84	50.24	47.48	41.60
bp230	0.47M	52.28	51.96	49.26	45.38	40.12

Table 10: ImageNet-100, retrained 120 epochs, magnitude pruning with a 30-epoch KD fine-tune, best over V. bp207 at 2:4 *exceeds* its dense accuracy: the distilled fine-tune acts as extra training.

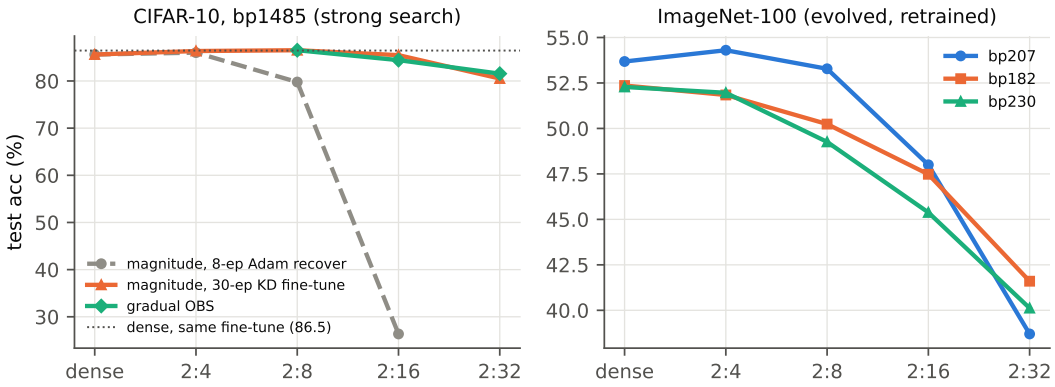


Figure 17: Left: CIFAR-10 winner bp1485 under the three recovery schemes (best V; dashed: the 8-epoch recipe of the original study; dotted: the dense model with the same 30-epoch fine-tune). Right: ImageNet-100, which degrades gracefully to 38.7–41.6% at 2:32.

The small CIFAR-10 search of our own (bp495 / bp671 / bp546, CoDeepNEAT weights, 8-epoch fine-tune) is in Appendix D.

5.9 Speed

GEMM level. Each pruned conv is timed as an im2col GEMM, Spatha against a dense fp16 GEMM of the same shape, excluding im2col itself.

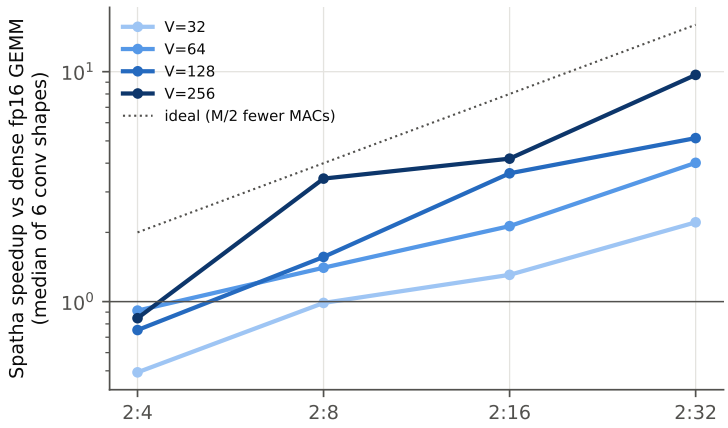


Figure 18: Spatha SpMM speedup on six typical conv GEMM shapes (batch 128). The speedup grows with V and M. At 2:4 Spatha is *slower* than dense for most V.

	$V = 32$				$V = 64$				$V = 128$			
model	2:4	2:8	2:16	2:32	2:4	2:8	2:16	2:32	2:4	2:8	2:16	2:32
bp493 (res)	0.95	1.43	2.12	3.84	1.16	1.77	2.81	5.44	1.10	1.83	3.66	7.88
bp741 (res)	0.93	1.56	2.32	3.66	1.18	2.11	3.14	4.75	1.22	2.06	3.55	5.84
bp482 (res)	0.89	1.77	5.37	14.0†	1.40	1.98	5.95	16.9†	1.66	2.57	7.35	29.6†
bp430 (orig.)	0.85	1.68	3.11	5.68	1.19	2.16	3.67	9.66	1.26	2.50	5.37	19.1†
r18	0.65	1.23	13.3†	23.1†	0.86	4.12	15.5†	25.8†	0.95	2.23	14.8†	25.4†
r56w32	0.08	0.93	1.47	1.93	0.07	1.06	1.65	1.65	0.08	0.24	1.27	1.65
ImageNet-100 bp207	–	–	–	–	0.98	1.49	2.39	3.64	0.85	1.54	2.51	3.44

Table 11: Summed conv-GEMM speedup (batch 128). † Above the ideal $M/2$ reduction, so inflated by an unstable dense baseline on the shared GPUs: read as "large". Narrow ResNet-56 (32-channel layers padded to V) is launch-bound and does not speed up.

End to end. Every conv of the network runs through `SpathaConv2d`, and the whole forward pass is timed. Speedups are against *dense-best*: per layer, the faster of cuDNN and the same im2col path with a dense GEMM. This isolates what sparsity adds.

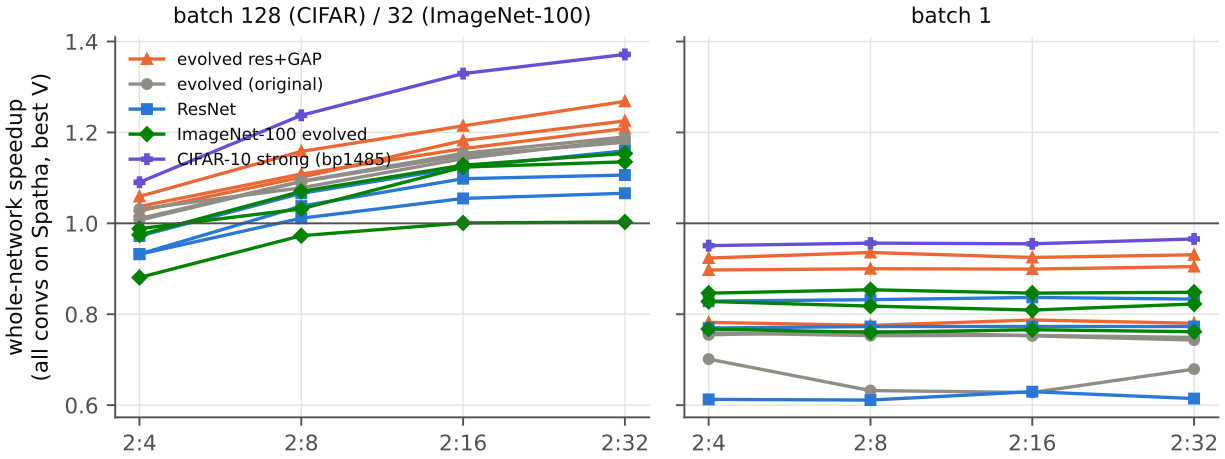


Figure 19: Whole-network inference speedup with every conv on Spatha (best V). At large batch the gain grows with M but stays below 1.4×; the CIFAR-10 winner bp1485 (widest convs) gains most. At batch 1 Spatha is slower (overhead-bound).

dataset	model	dense-best ms	all-Spatha 2:4	all-Spatha 2:8	all-Spatha 2:16	all-Spatha 2:32	hybrid (per-layer fastest) 2:4	hybrid (per-layer fastest) 2:8	hybrid (per-layer fastest) 2:16	hybrid (per-layer fastest) 2:32
CIFAR-100	bp493 (res)	24.3	1.03	1.10	1.18	1.23	1.03	1.09	1.17	1.21
CIFAR-100	bp741 (res)	16.4	1.06	1.16	1.21	1.27	1.07	1.16	1.22	1.27
CIFAR-100	bp482 (res)	6.3	1.04	1.11	1.16	1.21	1.05	1.11	1.17	1.21
CIFAR-100	bp430 (orig.)	7.4	1.01	1.09	1.15	1.18	1.07	1.16	1.21	1.23
CIFAR-100	bp427 (orig.)	7.3	1.03	1.08	1.14	1.18	1.07	1.14	1.22	1.26
CIFAR-100	bp499 (orig.)	7.2	1.01	1.09	1.15	1.19	1.10	1.15	1.23	1.24
CIFAR-100	r18	17.7	0.97	1.07	1.12	1.16	1.00	1.07	1.12	1.14
CIFAR-100	r20w58	21.7	0.93	1.04	1.10	1.11	1.00	1.03	1.09	1.09
CIFAR-100	r56w32	29.6	0.93	1.01	1.05	1.07	0.99	1.02	1.05	1.07
CIFAR-10	bp1485 (strong)	22.6	1.09	1.24	1.33	**1.37**	1.09	1.25	1.33	**1.37**
ImageNet-100	bp207	14.4	0.97	1.07	1.13	1.15	1.01	1.08	1.14	1.17
ImageNet-100	bp182	12.5	0.99	1.03	1.12	1.14	1.03	1.07	1.13	1.18
ImageNet-100	bp230	1.6	0.88	0.97	1.00	1.00	1.01	1.01	1.01	1.01
all models, batch 1			0.50–0.94				0.85–1.03			

Table 12: End-to-end speedup of a full forward pass (best over $V \in \{32, 64, 128\}$) relative to dense-best, at batch 128 (CIFAR) or 32 (ImageNet-100). Medians of 5 interleaved rounds; median round-to-round spread 2%. All 312 runs: `results/end_to_end/e2e.csv`. bp1485 was timed later, while our own training jobs shared the GPUs.

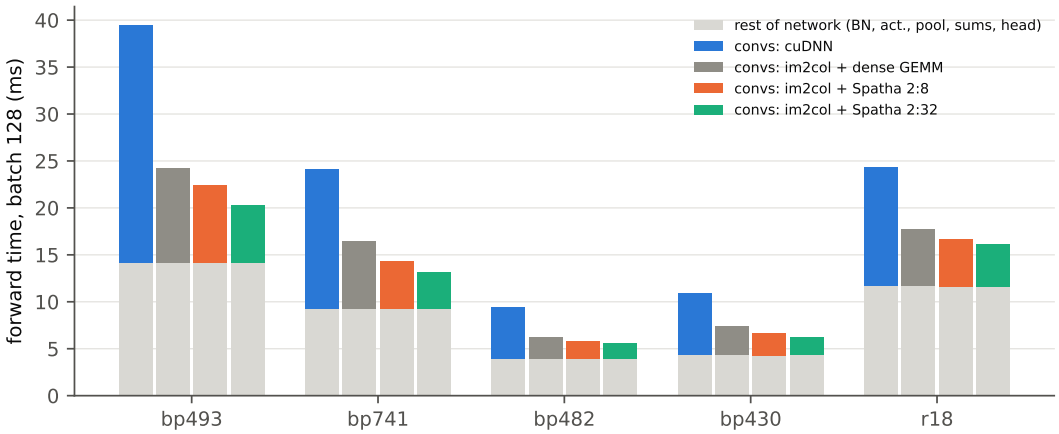


Figure 20: Where the time goes (V=128, batch 128). The grey base is everything that is not a convolution; on top is the conv time of each implementation. Convolutions are only 34–42% of the dense-best forward pass. Within them, im2col costs about as much as the sparse GEMM: for bp493 at 2:32 the GEMM is about 8× faster, but im2col + Spatha is only 1.6× faster than im2col + dense GEMM.

Speed is limited by Amdahl's law, applied twice. Convolutions are under half of the forward time (60% at most, for the CIFAR-10 winner), and within each convolution the im2col copy is as expensive as the sparse GEMM. A 4–8× GEMM speedup therefore becomes a 1.1–1.37× network speedup. Evolved models gain more than ResNets (1.27–1.37× vs. 1.16×) because their wide, large-kernel convs make the GEMM a larger share of the conv.

5.10 Comparison with hand-designed ResNets (brief)

As a sanity check, we trained CIFAR ResNets from scratch with the identical recipe and pruned them identically. Figure 21 summarises the comparison.

- **Dense accuracy:** ResNet-20/56 are 1.6–1.8 pt more accurate than the best evolved model at under half its size; ResNet-18 is 3.3 pt more accurate and larger.
- **2:4 to 2:16:** under the same magnitude pruning, the *drop* from dense is the same for both families. Evolved topologies are not harder to sparsify.
- **2:32:** gradual OBS on the evolved models (63.7%) beats every magnitude-pruned ResNet (best 54.7%). OBS was not run on the ResNets, so this compares criteria as much as architectures.

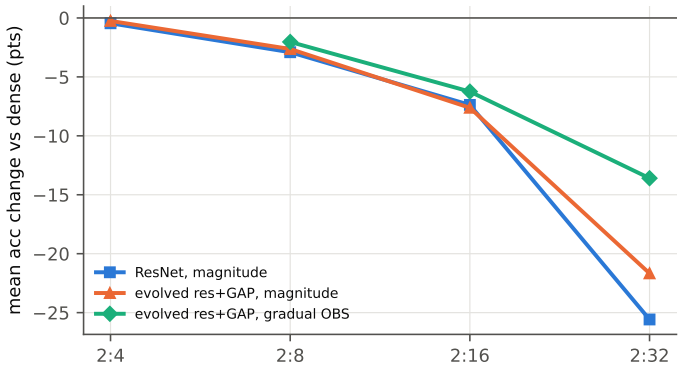


Figure 21: Mean accuracy change against dense (best V, 3 models per family).

6 Was V:N:M useful?

Yes, as compression; only modestly as acceleration.

goal	configuration (bp493, CIFAR-100)	acc.	conv comp.	stored	conv GEMM speedup	end-to-end (bs 128)
dense reference	–	76.50	1×	33.7 MB	–	–
near-lossless	V=8, 2:4, magnitude	76.26	3.1×	10.8 MB	– (V=8)	–
balanced, Spatha-runnable	V=32, 2:8, magnitude	74.01	6.4×	5.4 MB	1.43×	1.08×
best 2:8	V=8, 2:8, gradual OBS	74.40	6.3×	5.4 MB	–	–
high compression	V=8, 2:16, gradual OBS	70.50	12.6×	2.8 MB	–	–
max compression	V=8, 2:32, gradual OBS	**63.72**	25.1×	**1.4 MB**	–	–
max comp., Spatha-runnable	V=32, 2:32, gradual OBS	61.50	25.5×	≈1.4 MB	3.84×	1.19×

Table 13: Recommended operating points for the best evolved CIFAR-100 model. Speedups are measured on the same (V, M) pattern; kernel time depends only on the pattern, not on the weight values.

goal	configuration (bp1485, CIFAR-10)	acc.	conv comp.	stored	end-to-end (bs 128)
dense reference	30-ep fine-tuned, no pruning (control)	86.45	1×	44.4 MB	–
lossless	V=8, 2:8, magnitude, 30-ep KD	**86.52**	6.3×	7.0 MB	–
lossless, Spatha-runnable	V=32, 2:8, magnitude, 30-ep KD	86.36	6.6×	6.7 MB	1.11×
high compression	V=8, 2:16, magnitude, 30-ep KD	85.48	12.6×	**3.5 MB**	–
high comp., Spatha-runnable	V=32, 2:16, magnitude, 30-ep KD	85.02	13.2×	3.4 MB	1.25×
max compression	V=32, 2:32, magnitude, 30-ep KD	80.52	26.3×	1.7 MB	**1.30**×
max compression, best accuracy	V=8, 2:32, gradual OBS	81.55	25.3×	1.8 MB	–

Table 14: Operating points for the CIFAR-10 winner (dense checkpoint 85.61%, 44.4 MB fp32). End-to-end speedup with every conv on Spatha, against the fastest dense implementation of each layer.

- **Storage.** On CIFAR-100 a 33.7 MB evolved CNN becomes 5.4 MB with 2.1 pt loss, or 1.4 MB with 12.8 pt loss; this is only possible because the residual+GAP search space puts the parameters into convolutions. On CIFAR-10 a 44.4 MB evolved CNN becomes 7.0 MB with no loss and 3.5 MB with 1 pt loss.
- **Executability.** Every pruned network runs on Sparse Tensor Cores through Spatha and matches cuDNN to fp16 precision.
- **Speed.** The sparse GEMMs are 1.4–8× faster, but whole networks are only 1.1–1.37× faster at large batch and not faster at batch 1.

7 Novel findings

1. **V:N:M transfers to CNNs through the $(k_h, k_w, C_{\text{in}})$ im2col layout.**
   - Across 15 well-trained CNNs on three datasets, 2:4 loses a median 0.3 pt.
   - 2:8 loses 2–4 pt on CIFAR-100 and 0.4–3 pt on ImageNet-100, and nothing on the wide CIFAR-10 winner, which loses 1 pt at 2:16.
   - In this layout, the vector step removes (tap, input-channel) pairs for groups of output channels, and convolutions tolerate that well.

2. **The NAS search space decides whether V:N:M pays off.**
   - The default CoDeepNEAT space leaves 34–77% of the bytes in an unprunable flatten+linear head, which caps whole-model compression at about 1.3–2×.
   - Residual-sum aggregation plus GAP moves 99%+ of the bytes into convs, giving 24× whole-model compression. It also raised dense accuracy by 4 pt.
   - Searching for sparse deployment needs a sparsity-friendly space, not just a better pruner.
3. **Irregular evolved widths clash with large V.**
   - Row padding to a multiple of V cuts 2:8 compression from 6.3× to 4.1–5.7× at V=128 on evolved models.
   - On power-of-two ResNet widths, larger V instead *improves* compression, to 7.0×, through shared column ids.
   - A V:N:M-aware search should constrain channel counts to multiples of V.
4. **Accuracy and speed favour opposite V.** Accuracy is best at V=8. Spatha exists only for $V \geq 32$ and is fastest at V=128. The cost of V=128 is 1–3 pt at 2:8 and 3–5 pt at 2:16.
5. **Gradual second-order pruning is what makes high V:N:M sparsity usable on the CIFAR-100 models.**
   - It adds 9.2 pt on average at 2:32 and wins 26 of 27 comparisons.
   - One-shot OBS improves zero-shot accuracy by up to 48 pt, but that gain mostly disappears after fine-tuning. Zero-shot accuracy is therefore a poor proxy for the final result.
   - In the gradual schedule, removing the first half of the columns (vector-only sparsity) costs only about 2 pt, and the final 2:4 step with a long fine-tune recovers accuracy.
   - How much it helps depends on the network: on the wide CIFAR-10 winner it adds only 2.4 pt at 2:32 and nothing below that.
6. **Evolved CNNs are more Spatha-friendly than deep thin ResNets.**
   - Short-budget evolution produces shallow, wide, large-kernel networks (5×5 and 7×7, 142–342 channels; up to 1248 on CIFAR-10).
   - Their GEMMs are large, so they gain the most end to end: up to 1.27× on CIFAR-100 and 1.37× for the CIFAR-10 winner (2:32, V=128), whose convs are 60% of its forward time, against 34–42% for the other models.
   - ResNet-56 with 32-channel layers does not speed up at all.
7. **Two silent failure modes of Spatha.**
   - At M=32, K must be padded to a multiple of 256, otherwise there is an out-of-bounds write.
   - One call is limited to 65,535 column tiles, beyond which it returns garbage.

   Neither is documented, and both corrupt results or later kernels without any error.
8. **The recovery recipe decides what looks feasible.** On the same CIFAR-10 checkpoint, an 8-epoch Adam recover without augmentation or distillation gives 26% at 2:16; our 30-epoch distilled fine-tune with crop and flip gives 85.5%, 1 pt below a dense model fine-tuned the same way. Because such a fine-tune also improves the dense model (85.6 → 86.5%), V:N:M results should be compared against an equally fine-tuned dense control.
9. **CoDeepNEAT's evaluator overstates accuracy by $n/(n-1)$** for n test batches (it divides the summed batch accuracies by the last batch index): +1.3% relative at batch 128 on CIFAR, +2.6% at 256. Published or reported CoDeepNEAT accuracies computed with it should be read with this in mind. It is fixed in our copy.
10. **Training dominates search for the final accuracy.** The same evolved architectures went from at most 50% to 72.4% on CIFAR-100 (and from 28.9% to 52.3% on ImageNet-100) by retraining alone. Pruning studies on under-trained evolved models (Appendix D) understate how much V:N:M can retain.

8 Limitations

- **Single seed, test-set selection.** Each cell is one run, and "best over V" and the top models were picked on the test set. Differences under about 0.5 pt are not significant.
- **Unequal fine-tune budget.** Gradual OBS had 35–45 epochs against 30.
  - One-shot OBS and magnitude pruning, at equal budget, are within 0.3 pt of each other at 2:8 and 2:16, so we believe the extra epochs are not the main cause of the 2:32 gain.
  - The definitive control, gradual *magnitude* pruning on the same schedule, was not run.
- **OBS was not applied to the ResNets.**
- **Timing.**
  - GPUs were shared and fully loaded.
  - Execution was PyTorch 1.13 eager mode, without operator fusion or CUDA graphs.
  - V=8/16 settings cannot be timed.
  - GEMM-level numbers marked † exceed the theoretical bound.
- **Search.** Our own searches were short (10–12 generations); only the strong CIFAR-10 search ran 80 generations. No search used augmentation or a VENOM-aware fitness.
- **CIFAR-10 winner.** Only one model (bp1485) went through the full pipeline; its dense weights come from CoDeepNEAT's training without augmentation, which is why the dense control is needed.
- **Literature.** We found no prior V:N:M results on CNNs (VENOM evaluates transformers), but we did not do a systematic search.

9 Future work

- Gradual magnitude pruning as a control; gradual OBS on the ResNets; multiple seeds with a validation split.
- A VENOM-aware search:
  - score candidates *after* V:N:M pruning, or add size/latency as an objective;
  - restrict channel counts to multiples of V;
  - use longer evaluations so deeper networks can win.
- An implicit-GEMM Spatha convolution (no im2col), fused BN/activation epilogues, and V=8/16 kernels, followed by new end-to-end measurements.
- Full ImageNet.

10 Conclusion

V:N:M sparsity applies cleanly to CNNs, including the irregular networks found by CoDeepNEAT, provided two things hold:

- the search space puts the parameters into convolutions (residual sum + GAP);
- high sparsity is reached gradually with second-order pruning.

The best evolved CIFAR-100 model goes from 33.7 MB to 5.4 MB at 74.4% (2:8) and to 1.4 MB at 63.7% (2:32). The CIFAR-10 winner of an 80-generation search goes from 44.4 MB to 7.0 MB with no loss and to 3.5 MB at 85.5% (2:16), once it is fine-tuned properly. Every pruned network runs on Sparse Tensor Cores. The remaining gap is inference speed: turning large sparse-GEMM speedups into network speedups needs a sparse convolution kernel that avoids im2col.

A Per-run comparison of pruning criteria

bp	V	M	magnitude	OBS one-shot	OBS gradual	grad.−mag.	grad.−1-shot
482	8	2:8	69.60	69.94	70.24	+0.64	+0.30
482	8	2:16	62.14	62.75	64.33	+2.19	+1.58
482	8	2:32	42.37	47.23	55.37	+13.00	+8.14
482	32	2:8	68.94	69.33	69.58	+0.64	+0.25
482	32	2:16	59.02	58.44	63.65	+4.63	+5.21
482	32	2:32	41.95	42.03	51.49	+9.54	+9.46
482	128	2:8	67.99	68.47	68.99	+1.00	+0.52
482	128	2:16	56.95	56.59	61.89	+4.94	+5.30
482	128	2:32	38.67	39.14	48.89	+10.22	+9.75
493	8	2:8	73.97	73.92	74.40	+0.43	+0.48
493	8	2:16	70.06	69.23	70.50	+0.44	+1.27
493	8	2:32	57.23	57.98	63.72	+6.49	+5.74
493	32	2:8	74.01	73.75	73.83	−0.18	+0.08
493	32	2:16	67.69	67.76	69.49	+1.80	+1.73
493	32	2:32	52.79	52.38	61.50	+8.71	+9.12
493	128	2:8	72.97	73.26	73.95	+0.98	+0.69
493	128	2:16	66.34	65.46	68.57	+2.23	+3.11
493	128	2:32	48.94	51.13	60.14	+11.20	+9.01
741	8	2:8	70.15	70.45	70.92	+0.77	+0.47
741	8	2:16	66.60	66.43	68.09	+1.49	+1.66
741	8	2:32	57.05	57.11	61.78	+4.73	+4.67
741	32	2:8	69.71	70.03	70.10	+0.39	+0.07
741	32	2:16	64.85	64.72	66.86	+2.01	+2.14
741	32	2:32	51.85	52.85	59.91	+8.06	+7.06
741	128	2:8	68.96	69.46	69.90	+0.94	+0.44
741	128	2:16	63.29	62.89	65.90	+2.61	+3.01
741	128	2:32	47.20	51.48	57.89	+10.69	+6.41

Table 15: Every run of the criterion comparison (CIFAR-100, evolved residual+GAP models).

B Accuracy by V (CIFAR-100, magnitude)

	2:4					2:8				
model	V=8	16	32	64	128	8	16	32	64	128
bp493	76.26	76.09	76.03	75.84	76.21	73.97	73.89	74.01	73.09	72.97
bp741	72.20	72.15	72.14	71.93	71.91	70.15	69.80	69.71	69.18	68.96
bp482	72.24	72.36	72.44	72.34	72.35	69.60	69.43	68.94	68.11	67.99
bp430	71.18	71.30	71.34	71.34	71.10	68.40	67.77	67.17	65.97	66.10
bp427	71.07	70.86	71.50	70.70	71.07	68.61	67.99	67.12	65.93	65.39
bp499	69.24	69.29	69.18	69.25	69.26	66.84	66.83	66.84	65.71	66.02
r18	79.18	79.42	79.49	79.44	79.48	77.62	77.12	77.08	76.17	75.82
r20w58	77.68	77.75	77.82	78.13	77.83	75.43	75.19	74.66	74.16	74.33
r56w32	77.08	77.20	77.12	76.83	77.14	74.40	73.45	73.60	72.67	73.75

Table 16: At 2:4 the spread across V is at most 0.8 pt. At 2:8, V=8 is best, or within 0.05 pt of best, for all 9 models.

C ResNet results (CIFAR-100, magnitude)

model	params	dense	2:4	2:8	2:16	2:32
ResNet-18	11.22M	79.83	79.49	77.62	73.06	54.72
ResNet-20 (w58)	3.58M	78.26	78.13	75.43	70.79	50.63
ResNet-56 (w32)	3.42M	78.08	77.20	74.40	70.19	54.08

Table 17: Trained from scratch with recipe v2 and pruned like the evolved models (best over V).

D Earlier runs on under-trained weights

dataset	weights / fine-tune	model	params	dense	2:4	2:8	2:16	2:32
CIFAR-10[c]	CoDeepNEAT 60 ep / 8 ep	bp495	1.91M	80.60	80.28	79.03	71.68	34.62
CIFAR-10[c]	CoDeepNEAT 60 ep / 8 ep	bp671	0.57M	79.73	79.75	73.15	53.92	39.16
CIFAR-10[c]	CoDeepNEAT 60 ep / 8 ep	bp546	2.84M	78.26	77.54	71.88	47.72	31.48
CIFAR-100[c]	CoDeepNEAT 60 ep / 8 ep	bp606	1.22M	49.20	48.59	41.60	15.21	6.66
CIFAR-100[c]	CoDeepNEAT 60 ep / 8 ep	bp315	4.30M	44.27	43.69	40.48	26.94	11.11
CIFAR-100[c]	CoDeepNEAT 60 ep / 8 ep	bp735	38.19M	41.85	39.99	34.82	25.83	19.37
CIFAR-100	recipe v1, 200 ep / 30 ep KD	bp427	3.27M	67.50	67.42	65.33	60.00	43.53
CIFAR-100	recipe v1, 200 ep / 30 ep KD	bp430	3.62M	67.44	67.06	64.63	59.62	47.36
CIFAR-100	recipe v1, 200 ep / 30 ep KD	bp499	5.49M	67.06	67.00	64.90	61.13	53.24
ImageNet-100[c]	CoDeepNEAT 40 ep / 3 ep	bp230	0.40M	28.89	28.77	15.38	5.94	3.34
ImageNet-100[c]	CoDeepNEAT 40 ep / 3 ep	bp254	0.40M	27.74	26.56	16.35	5.74	2.55

Table 18: Magnitude pruning, best over V, on weights from CoDeepNEAT's own training with its short fine-tune (CIFAR-10: our 12-generation search). [c] Corrected for the evaluator bug (Section 4.3). A third ImageNet-100 model from this stage (bp22) had a broken checkpoint and is omitted. Even on these weak models, 2:4 is near-lossless.

E Changes made to CoDeepNEAT

- `configuration.py`: new options `global_avg_pool`, `residual_sum`, `final_relu` and `image_size`.
- `neural_network.py`: GAP head, optional logit ReLU, and `.view`→`.reshape` (needed for channels-last tensors).
- `conv_homogeniser.py`: residual-sum aggregation (identity for the widest input, 1×1 projection for the others).
- `conv_linear_homogeniser.py`, `reshape.py`: `.reshape` for channels-last tensors.
- `data_loader.py`: CIFAR-100 and ImageNet-100 loaders, plus the `image_size` option.
- `phenotype_evaluator.py`, `generation.py`: drop blueprints whose classifier would be too large, instead of crashing.
- `augmentations/*`: `imgaug` compatibility with current NumPy.
- `evaluator.py`: `test_nn` counts every sample once (Section 4.3).

F Reproduction

step	command (from the repository root)
setup (envs, Spatha, data)	`scripts/setup_main_env.sh`; `scripts/setup_venom.sh`; `scripts/setup_data.sh all`
everything, in order	`scripts/run_all.sh` (stages: setup, search, train, prune, speed, figures)
evolve + CoDeepNEAT full train	`scripts/run_search.sh <config>`, e.g. `venom_cifar100_res`, `ada_cifar10_strong`
retrain (recipe v2)	`scripts/run_retrain_v2.sh res` (`orig` for the original space)
ImageNet-100 data	`scripts/export_imagenet100.py`; `vnm/build_cache.py`
ResNet baselines	`scripts/run_resnets.sh`
$V \times M$ grid, magnitude	`scripts/run_gridres.sh` (also `run_grid3.sh`, `run_grid_resnet.sh`)
one-shot / gradual OBS	`scripts/run_obs.sh`, `scripts/run_gradual.sh`
CIFAR-10 winner bp1485	`scripts/run_cifar10_ada.sh` (8-ep recipe, 30-ep grid, dense control, gradual OBS)
architecture graphs	`vnm/plot_architectures.py` (CoDeepNEAT's own visualisers)
Spatha GEMM timing	`vnm/spatha_bench.py` (run by the grid scripts); `scripts/run_vscan.sh <gpu> <V>`
end-to-end timing	`scripts/run_e2e.sh <gpu> <V>`; ImageNet-100 check: `scripts/run_in100_check.sh`
raw results	`CoDeepNEAT/runs/<run>/<grid>/results.jsonl`, `results/`

References

[1] R. Miikkulainen et al., "Evolving deep neural networks," in *Artificial Intelligence in the Age of Neural Networks and Brain Computing*, 2019 (arXiv:1703.00548).

[2] R. L. Castro, A. Ivanov, D. Andrade, T. Ben-Nun, B. B. Fraguela, T. Hoefler, "VENOM: A vectorized N:M format for unleashing the power of sparse tensor cores," *SC'23*.

[3] B. Hassibi, D. G. Stork, "Second order derivatives for network pruning: Optimal Brain Surgeon," *NeurIPS*, 1993.

[4] E. Frantar, D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," *ICML*, 2023.

[5] E. Kurtic et al., "The Optimal BERT Surgeon: Scalable and accurate second-order pruning for large language models," *EMNLP*, 2022.

[6] Y. Tian, D. Krishnan, P. Isola, "Contrastive multiview coding," *ECCV*, 2020.

[7] A. Mishra et al., "Accelerating sparse deep neural networks," arXiv:2104.08378, 2021.

[8] A. Zhou et al., "Learning N:M fine-grained structured sparse neural networks from scratch," *ICLR*, 2021.